

Assalam o alaikum, Mam I have implemented design code for all 3 qs, and also write testbenches file , including self checking testbenches and requirements you mentioned

```
module Task1Q1(output [31:0] sum,output cout,  
    input [31:0] a,  
    input [31:0] b, input cin);
```

```
    assign {cout,sum} = a + b+cin;  
endmodule
```

```
module Task1Q1_tb;  
reg [31:0] a;  
reg [31:0] b;  
reg cin;  
wire [31:0] sum;  
wire cout;
```

```
Task1Q1 uut (  
    .a(a),  
    .b(b),  
    .cin(cin),  
    .sum(sum),  
    .cout(cout)  
);  
integer pass, fail;  
reg [32:0] expected;
```

```
initial begin  
    pass = 0; fail = 0;
```

```
    // Directed test  
    a = 0; b = 0; cin = 0;  
    #1;  
    expected = a + b + cin;  
    if ({cout,sum} == expected) pass++;  
    else fail++;
```

```
    // Random tests  
    repeat (100) begin  
        a = $random;  
        b = $random;  
        cin = $random;  
        #1;  
        expected = a + b + cin;  
        if ({cout,sum} == expected) pass++;  
        else fail++;  
    end
```

```

    $display("PASS=%0d FAIL=%0d", pass, fail);
    $finish;
end
endmodule

```

```

//Barrel Shifter module
module Task1Q2(output reg [31:0] data_out,
    input [31:0] data_in,
    input [4:0] shift_amt, //shift amount =5 bits for 32 bits input
    input dir); //direction: 0 for left, 1 for right

always @(*) begin
    if(shift_amt == 0) begin
        data_out = data_in; //no shift
    end else if (dir == 0) begin
        data_out = (data_in << shift_amt) | (data_in >> (32 - shift_amt)); //rotate left shift
    end else begin
        data_out = (data_in >> shift_amt) | (data_in << (32 - shift_amt)); //rotate right shift
    end
end
endmodule

```

```

`timescale 1ps/
module Task1Q2_tb;
reg [31:0] data_in;
reg shift_amt;
reg dir;
wire [31:0] data_out;
integer pass=0, fail=0;
reg [31:0] expected;

Task1Q2 uut( .data_in(data_in), .data_out(data_out), .shift_amt(shift_amt), .dir(dir), );

task check();
if (data_out==expected) $display("PASS=%0d FAIL=%0d", pass++, fail);
else $display("PASS=%0d FAIL=%0d", pass, fail++);
endtask

initial begin
    #10;
    shift_amt =0;
    dir=$random(0,1);

```

```

data_in = $random;
expected = data_in; //no shift
check();
#10;
dir = 0; //left shift
shift_amt = $random(1,31);
data_in = $random;
expected= (data_in << shift_amt) | (data_in >> (32 - shift_amt)); //rotate left shift
check();
#10;
dir=1; //right shift
shift_amt = $random(1,31);
data_in = $random;
expected= (data_in >> shift_amt)|(data_in << (32 - shift_amt)); //rotate right shift
check();
#10;
$display("PASS=%0d FAIL=%0d", pass, fail);
$finish;
end
endmodule

```

//priority enc ; 8 to 3 encoder

```

module Task1Q3(input [7:0]in, output reg[2:0]out, output reg valid);

```

```

always@(*)begin
casez(in):
8'b00000001: begin out=3'b000; valid=1'b1; end
8'b0000001?: begin out=3'b001; valid=1'b1; end
8'b000001??: begin out=3'b010; valid=1'b1; end
8'b00001???: begin out=3'b011; valid=1'b1;
end
8'b0001?????: begin out=3'b100; valid=1'b1;end
8'b001?????: begin out=3'b101; valid=1'b1; end
8'b01?????: begin out=3'b110; valid=1'b1;
end
8'b1?????: begin out=3'b111; valid=1'b1; end
default: begin out=3'b000; valid=1'b0; end
endcase
end
endmodule
`timescale 1ns/1ps

```

```

module Task1Q3_tb;

```

```

    reg [7:0] in;

```

```
wire [2:0] out;  
wire    valid;
```

```
reg [2:0] expected_out;  
reg    expected_valid;  
integer pass, fail;  
integer i;
```

```
Task1Q3 uut (  
    .in(in),  
    .out(out),  
    .valid(valid)  
);
```

```
task check;  
    begin  
        if (out === expected_out && valid === expected_valid)  
            pass = pass + 1;  
        else  
            fail = fail + 1;  
    end  
endtask
```

```
initial begin  
    pass = 0;  
    fail = 0;
```

```
    // Valid one-hot tests  
    for (i = 0; i < 8; i = i + 1) begin  
        in = 8'b1 << i;  
        #1;  
        expected_out = i[2:0];  
        expected_valid = 1'b1;  
        check();  
    end
```

```
    // Invalid: no bits high  
    in = 8'b00000000;  
    #1;  
    expected_out = 3'b000;  
    expected_valid = 1'b0;  
    check();
```

```
    // Invalid: multiple bits high  
    in = 8'b00101000; // multiple 1s  
    #1;  
    expected_out = 3'd5; // highest bit position
```

```
expected_valid = 1'b1;  
check();  
  
$display("FINAL RESULT: PASS=%0d FAIL=%0d", pass, fail);  
$finish;  
end
```

endmodule

for the simulations, i try to simulate them using quartus prime, There's no error in synthesis and simulation but no outputs are seen on timing diagram, i follow multiple tutorials but i think I'm missing some step as I'm using quartus prime for the first time, I'll try to search and practice how to use it more